

Verteilte Systeme Übungsblatt 3

Aufgabe 1

MongoDB ist ein dokumentenorientiertes NoSQL-Datenbanksystem, das seit 2007 vom US-amerikanischen Unternehmen MongoDB Inc. (vormals 10gen) entwickelt und vertrieben wird. Einer Datenerhebung der Webseite db-engines.com zufolge (betrieben von Redgate Software) ist MongoDB auf Platz eins im Popularitätsranking non-relationaler Datenbanken und auf Platz fünf insgesamt, weshalb ich es für dieses Übungsblatt ausgewählt habe.

Als NoSQL-Datenbank ist MongoDB einfach horizontal skalierbar. Die Datenbank setzt auf ein Sharding-System, bei dem die Daten in Partitionen – sogenannte Chunks – unterteilt und diese auf verschiedenen Shard-Servern abgelegt werden. Auf diese Art lässt sich durch Hinzufügen weiterer Shards auf einfache Art Kapazität sowohl für große Datenmengen als auch für Lasten schaffen, um so praktisch unbegrenzt horizontal zu skalieren.

Im Fall vom MongoDB, das als dokumentenorientiertes System mit JSON-ähnlichen Dateien arbeitet, wird jedem dieser Dokumente ein Shard-Key zugeordnet. Das Dokument wird dem Hash des Keys entsprechend dem zuständigen Chunk übergeben. Standardmäßig haben diese eine Maximalgröße von 64 MB. Wächst ein Chunk über diese Größe hinaus, wird er im Hintergrund automatisch geteilt. Ein Balancer verschiebt aktiv Chunks zwischen den Shards, bis alle ungefähr gleich viele Chunks enthalten. Das passiert asynchron im Hintergrund, was zu Performance-Peaks führen kann. Dies beeinträchtigt die Erreichbarkeit der Datenbank aber nicht. Shards sind üblicherweise als Replica-Sets aus mehreren Knoten konfiguriert, wobei ein Primary die Schreiboperationen abhandelt und Secondaries asynchron replizieren und je nach Konfiguration Leseanfragen bedienen.

Ein separates Replica-Set aus Config-Prozessen sammelt beim Ablegen der Dokumente Metadaten, die in einer separaten Datenbank sogenannten Mongos vorliegen. Client-Anfragen richten sich immer an diese Mongos-Prozesse, die dann den zuständigen Shard bestimmen und die Query entweder weiterleiten oder – wenn mehrere Shards betroffen sind – selbst abfragen und die Ergebnisse direkt zusammenfassen.

Wie bereits erwähnt, können die Knoten auf verschiedene Arten konfiguriert werden. Dies sorgt wahlweise für schnellere Antwortzeiten oder höhere Konsistenzen. Mit Blick auf PACELC ist festzustellen, dass MongoDB auf Grund der Verwendung von Replica-Sets grundsätzlich partitionstolerant ist. Mit Blick auf Verfügbarkeit und Konsistenz ist das Verhalten konfigurierbar. Müssen Lese- und Schreibvorgänge von einer Mehrheit bestätigt werden (Write Concern $w=majority$), ist das System stark konsistent, kann bei Partition jedoch teilweise unverfügbar sein, wenn keine Mehrheit gebildet werden kann. Ist diese Mehrheitsbestätigung nicht nötig (Write Concern $w=1$) bleibt das System schreibfähig, allerdings auf Kosten von Daten, die auf einem isolierten Primary geschrieben wurden und verloren gehen können. Außerhalb einer Partition sorgt die Mehrheitsbestätigung hier für gute Konsistenz, der Verzicht für geringe Latenz.

MongoDB ist also nicht starr einzuordnen, sondern kann je nach Bedarf eingestellt werden.

Für Aufgaben wie Analytics und Logging kann eine Erhöhung des Durchsatzes auf Kosten der Konsistenz sinnvoll, im Finanzwesen sollte entgegengesetzt optimiert werden.

Im Normalbetrieb sorgt Sharding außerdem dafür, dass das Datenvolumen linear mit den Shards wächst. Dadurch kann kein einzelner Server überlasten und zum Flaschenhals werden. Genauso wird die Schreibe- und Leselast auf mehrere Hosts aufgeteilt, was in den meisten Szenarien ein ähnliches Ziel verfolgt – und erreicht. Zuletzt sorgen die Replica-Sets innerhalb der Shards für eine hohe Verfügbarkeit, mit Optionen für Recovery.

Aufgabe 2

Als Testanwendung wird mit Node.js und Express ein simpler Counter-Service implementiert. Der Programmcode liegt der Abgabe bei, weshalb die Beschreibung hier bewusst kurz gehalten wird. MongoDB läuft für diesen Test in einen Dockercontainer.

Es werden auf App-Seite 2 Endpunkte implementiert:

POST /increment: {counterId, delta} – inkrementiert den entsprechenden Zähler, um den in *delta* angegeben Wert.

GET /value?counterId – gibt den aktuellen Wert des entsprechenden Zähler zurück.

Mittels ENV-Parametern können während des automatisierten Tests Read- und WriteConcern angepasst werden, um Neustarts zu vermeiden.

Die angesprochenen automatisierten Test werden mit Autocannon ermöglicht. Es ergeben sich 18 Szenarien à 30 Sekunden mit je 100 Connections. Die Szenarien setzen sich aus den Tests Read-Only, Write-Only und Mixed (80 Write / 20 Read) zusammen, kombiniert mit den drei Profilen für n=1, n=2 und n=3 Shards, sowie den zwei Konsistenzmodi PA/EL und PC/EC.

	PA/EL	PC/EC
Write Concern	1	Majority
Journaling	False	True
Read Concern	local	majority
Read Preference	primaryPreferred	primary

Tab.1 Attribute der Konsistenzmodi

Aufgabe 3

Ziel des Benchmarks ist es, zu analysieren wie sich horizontalen Skalierungen mit 1, 2 bzw. 3 Shards und die unterschiedlichen Konsistenzmodi auf Durchsatz und Latenz auswirken. Die Test sind einfach zu replizieren, da sie über ein Skript automatisiert ausgeführt werden. Der Mixed-Test ist mit einer 100-prozentigen Fehlerquote nicht aussagekräftig, weshalb die Daten hier nicht erneut aufgeführt werden. Folgende Messungen haben sich ergeben:

Shards	Durchsatz [ops/s]	Latenz p50 [ms]	Latenz p90 [ms]	Latenz p99 [ms]	Fehler
1	4138.97	221	334	482	0
2	4171.57	220	332	468	0
3	3810.20	241	369	530	0

Tab. 2 Ergebnisse Write-Only PA/EL

Shards	Durchsatz [ops/s]	Latenz p50 [ms]	Latenz p90 [ms]	Latenz p99 [ms]	Fehler
1	3437.77	268	396	552	0
2	3311.60	282	400	540	0
3	2985.14	310	450	650	0

Tab. 3 Ergebnisse Write-Only PC/EC

Shards	Durchsatz [ops/s]	Latenz p50 [ms]	Latenz p90 [ms]	Latenz p99 [ms]	Fehler
1	8296.34	58	73	109	0
2	7668.80	62	79	123	0
3	6897.47	70	86	130	0

Tab. 4 Ergebnisse Read-Only PA/EL

Shards	Durchsatz [ops/s]	Latenz p50 [ms]	Latenz p90 [ms]	Latenz p99 [ms]	Fehler
1	7783.60	62	77	105	0
2	7303.47	66	81	120	0
3	6849.70	70	88	141	0

Tab. 5 Ergebnisse Read-Only PC/EC

Durchweg zeigt sich, dass die Leseoperationen in allen Tests deutlich schneller abwickelt als die Schreiboperationen.

Bei den Write-Operationen fällt zudem auf, dass PC/EC den Durchsatz im Vergleich zu PA/EL um 15-30% senkt, zeitgleich aber die Latenz um 20-30% steigt, was den Erwartungen entspricht.

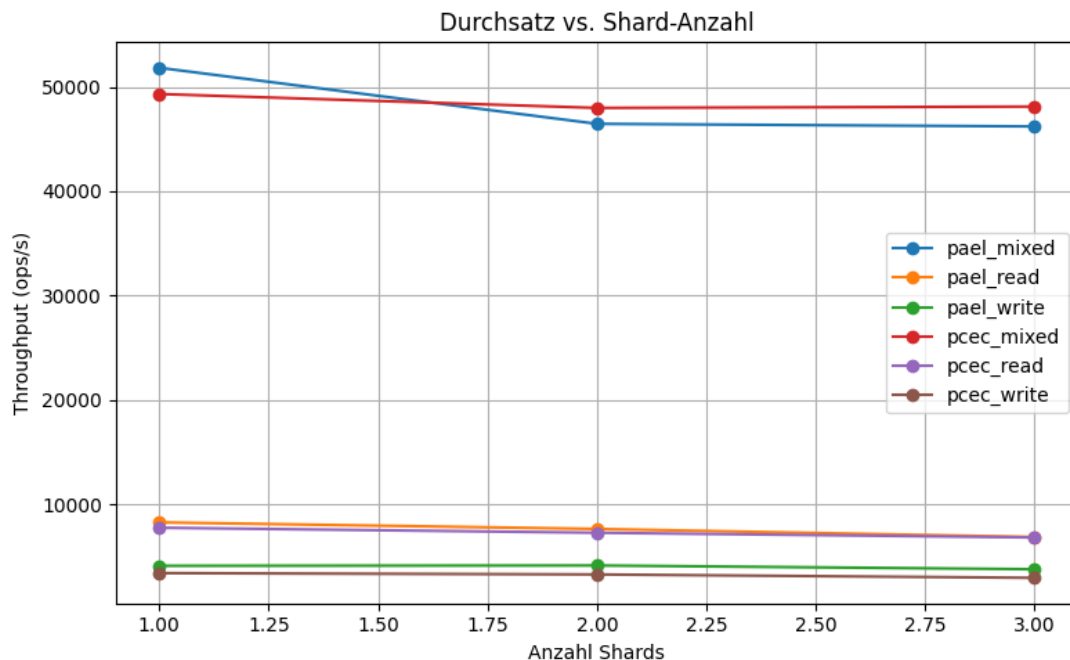


Fig. 1 Durchsatz vs. Shard-Anzahl

Entgegen der Erwartungen führen mehr Shards zu einem leicht verringerten Write-Durchsatz. Das ist darauf zurückzuführen, dass es nur einen Counter gibt, der in einem Document gespeichert ist. Dementsprechend greifen alle Operationen auf dieselben

Daten zu, weshalb mehr Shards kein Vorteil sind. Diese Absenz eines Skalierungsvorteils ist sowohl bei Read-Only als auch Write-Only zu sehen.

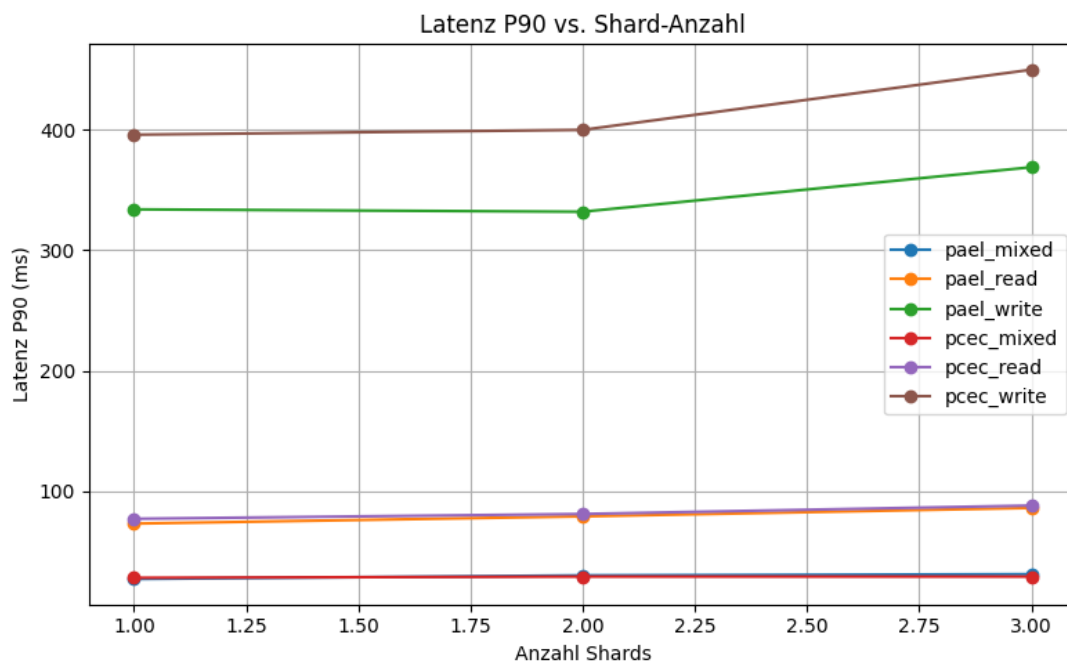


Fig. 2 Latenz p90 vs. Shard-Anzahl

Die Tests der Read-Operationen zeigen – ebenfalls den Erwartungen widersprechend – keinen signifikanten Unterschied in der Latenz zwischen PC/EC und PA/EL, wenngleich der Durchsatz leicht sinkt.

3.1 Best-Practice Empfehlungen

Es empfiehlt sich zunächst, das System der Shard-Key-Zuweisung weise zu wählen. In der Regel ist die Verwendung von Hashes einer starren Zuweisung vorzuziehen, je nach Anwendungsfall können aber andere Methoden vorteilhafter sein. Die Literatur empfiehlt beispielsweise Range-Keys für Range Queries.

Ebenso muss der Konsistenzmodus per Anwendungsfall gewählt werden. Wie bereits erwähnt, eignet sich ein PC/EC-System mit Majority-Read besonders für Systeme, die keine Fehler erlauben. Ist hingegen eine gewisse Toleranz akzeptabel, können andere Konfigurationen die Geschwindigkeit maßgeblich verbessern.

Zuletzt sollte auch die Skalierung beachtet werden. Single-Key-Workloads (wie im Test aus der vorherigen Textpassage) profitieren nicht von der Erhöhung der verwendeten

Shards; im Gegenteil dazu ist sogar ein Leistungsverlust möglich. Auch sollte darauf geachtet werden, rechenintensive Hintergrundprozesse für außerhalb der Stoßzeiten zu planen. Beispielsweise bietet es sich in der Regel an, Balancer-Jobs in der Nacht ausführen zu lassen, wenn weniger Systemzugriffe erwartet werden.